

Release Management — DocumentIA Production

Aplicable a: SRBRGDOCSAIPROD (Production Only) **Fuente:** Verificado contra pipelines + EF Core migrations

Tabla de Contenidos

- 1. Versioning Strategy
- 2. Release Cycle
- 3. Pre-Release Checklist
- 4. Release Process
- 5. Database Migrations
- 6. Rollback Procedures
- 7. Hotfix Process
- 8. Communication Plan
- 9. Post-Release Validation

Versioning Strategy

Semantic Versioning (SemVer)

Format: MAJOR.MINOR.PATCH-PRERELEASE+BUILD

Example: <major>.<minor>.<patch>-rc.<n>+<build-date>.<run>

Versioning Rules

Component	Increment When	Example
MAJOR	Breaking API change, major feature, critical vulnerability	1.0.0 → 2.0.0
MINOR	New feature, non-breaking enhancement, provider upgrade	1.4.5 → 1.5.0
PATCH	Bug fix, performance improvement, configuration change	1.4.5 → 1.4.6
PRERELEASE	RC/Beta/Alpha (before production release)	1.5.0-rc.1
BUILD	Build metadata (date + run number)	+

Version Placement

Functions App (src/backend/DocumentIA.Functions/)

```
<!-- DocumentIA.Functions.csproj -->
<PropertyGroup>
  <AssemblyVersion>MAJOR.MINOR.PATCH.0</AssemblyVersion>
  <FileVersion>MAJOR.MINOR.PATCH.0</FileVersion>
  <InformationalVersion>MAJOR.MINOR.PATCH-rc.N+BUILD</InformationalVersion>
</PropertyGroup>
```

Admin Web App (src/frontend/DocumentIA.Admin/)

```
<!-- DocumentIA.Admin.csproj -->
<PropertyGroup>
  <AssemblyVersion>MAJOR.MINOR.PATCH.0</AssemblyVersion>
  <FileVersion>MAJOR.MINOR.PATCH.0</FileVersion>
  <InformationalVersion>MAJOR.MINOR.PATCH-rc.N+BUILD</InformationalVersion>
</PropertyGroup>
```

AssetResolver Plugin (`src/plugins/DocumentIA.AssetResolver/`)

```
<!-- DocumentIA.AssetResolver.csproj -->
<PropertyGroup>
  <AssemblyVersion>MAJOR.MINOR.PATCH.0</AssemblyVersion>
  <FileVersion>MAJOR.MINOR.PATCH.0</FileVersion>
  <InformationalVersion>MAJOR.MINOR.PATCH-rc.N+BUILD</InformationalVersion>
</PropertyGroup>
```

Version Tracking

Azure DevOps / Git Tags:

```
# Tag format
vMAJOR.MINOR.PATCH-rc.N+BUILD

# Create tag
git tag -a vMAJOR.MINOR.PATCH-rc.N -m "Release MAJOR.MINOR.PATCH-rc.N: <resumen>"
git push origin vMAJOR.MINOR.PATCH-rc.N

# List tags
git tag -l "v1.*" | sort -V
```

Application Insights Annotation:

- Event created automatically when deployment completes
- Properties: version, environment, release_notes_link
- Used for correlation with performance metrics

Release Cycle

El ciclo de release sigue las fases: desarrollo y revisión de código → creación de Release Candidate (RC) → validación → aprobación → despliegue a producción → monitorización post-release. El detalle de cada fase está en la sección [Release Process](#).

Release Types

1. Standard Release (MINOR.PATCH bump)

- Approval: Tech Lead + Product Owner
- Rollback risk: Low

2. Hotfix Release (PATCH bump)

- Approval: Responsable técnico (revisión acelerada)
- Rollback risk: Medium

3. Major Release (MAJOR bump)

- Approval: Dirección técnica + revisión de arquitectura
- Rollback risk: High
- Incluye guía de migración

Pre-Release Checklist

Code Quality (Before RC)

- ☐ Automated Tests Pass

```
# Run test suite locally
dotnet test --configuration Release --verbosity minimal
```

- Unit tests: 100% pass
- Integration tests: 100% pass
- E2E smoke tests: 100% pass

- ☐ **Code Review Complete**

- All PRs approved (≥ 2 reviewers for MAJOR/hotfix, ≥ 1 for MINOR)
- All comments resolved
- Merge commits squashed

- ☐ **Linting & Analysis**

```
# Run code analysis
dotnet build /p:EnforceCodeStyleInBuild=true
# No warnings > Error level
```

- ☐ **Documentation Updated**

- README.md reflects changes
- Release notes prepared
- API docs updated (if applicable)

Dependency Updates (Before RC)

- ☐ **NuGet Packages**

```
# Check for security updates
dotnet list package --vulnerable
# No critical vulnerabilities
```

- ☐ **Runtime Versions**

- .NET version compatible (8 or 9)
- Azure Functions runtime check
- Durable Functions version validated

Infrastructure Readiness (Before RC)

- ☐ **Capacity Planning**

- Current CPU/Memory usage < 70%
- Storage capacity adequate
- Database connections available
- See `docs/infraestructura/INFRAESTRUCTURA_REAL_DESPLEGADA.md`

- ☐ **Monitoring Setup**

- Application Insights alerts configured
- Custom metrics ready
- Baseline metrics captured

- ☐ **Key Vault Secrets Current**

```
# Check secret expiration
az keyvault secret list --vault-name srbkvprodoci --query "[?attributes.expires < now_add('30d')].id" -o table
# Should be empty (no expiring secrets)
```

Security Review (Before RC)

- ☐ **No Hardcoded Credentials**

```
# Scan for secrets (last 20 commits)
git log -p -20 | Select-String -Pattern "password|secret|apikey" | Select-Object -Unique
# Should return nothing
```

- ☐ **RBAC Validated**

- Managed identity permissions correct
- No over-provisioned roles

- ☐ **Data Protection**

- Encryption at rest enabled
- Encryption in transit enforced
- No PII logged unencrypted

Database Readiness (Before RC)

- ☐ **Migration Script Tested**

```
# Test migration on local/test DB
dotnet ef database update --project src/backend/DocumentIA.Functions --startup-project src/backend/DocumentIA.Functions
# No errors
# Rollback tested via revert-migration script
```

- ☐ **Backup Created**

```
# SQL Server backup
az sql db backup create --server YOUR_SERVER --database DocumentIA --resource-group SRBRGDOCSAIPROD
# Backup verified by size > 0
```

Release Process

Phase 1: RC Creation

1. Create Release Branch

```
# Branch naming: release/v<version>
git checkout main
git pull origin main
git checkout -b release/v<version>
```

2. Update Version Numbers

```
<!-- All three apps (.csproj files): -->
<InformationalVersion>MAJOR.MINOR.PATCH-rc.N+BUILD</InformationalVersion>
```

3. Create Release Notes

```
# Release v<version>
## Features
- [<work-item>] Feature description

## Bug Fixes
- [<work-item>] Fix description

## Database Changes
- (columnas/índices añadidos, si aplica)

## Upgrade Path
1. Review breaking changes
2. Deploy new version
3. Run database migrations
4. Validate monitoring

## Known Issues
- (lista o "None")
```

4. Commit & Tag

```
git add .
git commit -m "Release v<version>: [release notes summary]"
git tag -a v<version> -m "Release candidate v<version>"
git push origin release/v<version>
git push origin v<version>
```

Phase 2: RC Validation

1. Run Automated Test Suite

- Pipeline runs automatically:
 - Unit tests (100% pass required)
 - Integration tests (100% pass required)
 - E2E smoke tests (100% pass required)
 - Security scanning (0 critical vulns)

2. Performance Comparison

```
// Run in AppInsights
customMetrics
| where timestamp between ((now(-1d)) .. (now()))
| where name == "DocumentIA.Duracion.Total"
| summarize p50_ms=percentile(value, 50), p95_ms=percentile(value, 95) by bin(timestamp, 1h)
| render timechart
```

3. Database Migration Test

```
# Test on backup restored to test environment
# 1. Restore backup
# 2. Run EF migrations
# 3. Verify data integrity (row counts, referential integrity)
# 4. Test rollback procedure
```

Phase 3: Release Approval

Approval Meeting

Attendees:

- Tech Lead (approval authority)
- Product Owner (business approval)
- On-Call Engineer (deployment lead)

Checklist:

- ☐ All automated tests passed
- ☐ Performance metrics acceptable
- ☐ Database migrations validated
- ☐ No regressions found
- ☐ Release notes complete
- ☐ Rollback plan reviewed

Approval Decision:

1. **Approve for Production** → Go to Phase 4
2. **Conditional Approval** → Fix issues → Re-test
3. **Reject** → Document reasons → Plan next release

Phase 4: Production Deployment

1. Pre-Deployment Communication (24h before)

```
**Deployment Notice**  
**Release:** v<version>  
**Start:** <fecha/hora> UTC  
**Duration:** 30-45 minutes  
**Impact:** Potential brief latency spikes (< 2 min)  
**Rollback:** Auto-enabled if errors  
**Lead:** <responsable on-call>
```

2. Pre-Deployment Validation (30 min before)

```
# 1. Verify Key Vault accessible  
az keyvault secret show --vault-name srbkvprodcai --name "AzureWebJobsStorage" --query "value" -o tsv | Measure-Object -  
Character  
  
# 2. Verify database backup completed  
az sql db backup list --server YOUR_SERVER --database DocumentIA --resource-group SRBRGDOCSAIPROD | Select-Object -First 1  
BackupTime  
  
# 3. Verify monitoring online  
az monitor app-insights app show --resource-group SRBRGDOCSAIPROD --app-insights-name srbappiprodcai --query "appId"  
  
# 4. Check resource capacity  
az functionapp plan show --resource-group SRBRGDOCSAIPROD --name YOUR_PLAN --query "Sku, NumberOfWorkers"  
  
Write-Output "All pre-deployment checks passed"
```

3. Execute Deployment Pipeline

```
# Via Azure DevOps UI:  
# 1. Navigate to Pipelines → azure-pipelines.yml  
# 2. Click "Run"  
# 3. Select parameter targetEnvironment: dev | pre | prod  
# 4. Click "Run"  
  
# Pipeline stages (azure-pipelines.yml):  
# Build & Test  
# RunMigrations (deshabilitado por defecto, condition: false)  
# DeployFunctions (incluye el deploy del Admin como job dentro de este stage)  
# DeployAssetResolver  
# ValidateConfiguration
```

4. Monitor Deployment

```
# Watch pipeline progress
$pipelineId = "YOUR_PIPELINE_RUN_ID"
while ($true) {
    $status = az pipelines runs show --id $pipelineId --query "status"
    $result = az pipelines runs show --id $pipelineId --query "result"
    Write-Output "Pipeline: $status | Result: $result"

    if ($status -in ("completed", "failed")) { break }
    Start-Sleep -Seconds 30
}
```

5. Post-Deployment Validation (15 min after)

```
# 1. Verify all apps running
az functionapp show --resource-group SRBRGDOCSAIPROD --name srbappprodoci --query "state"
# Expected: "Running"

# 2. Run smoke tests
./tests/smoke_e2e.ps1 -Environment Production

# 3. Check metrics (P95 latency)
# Query in AppInsights (latency check below)

Write-Output "Deployment successful. Monitoring enabled."
```

6. Metrics Check (AppInsights)

```
// Run immediately after deployment
customMetrics
| where timestamp > ago(10m)
| where name == "DocumentIA.Duracion.Total"
| summarize
    p50_ms=percentile(value, 50),
    p95_ms=percentile(value, 95),
    error_pct=sum(iff(value > 180000, 1, 0))/count()*100
    by bin(timestamp, 1m)
| render timechart
// Check: P95 < 60s, Error < 2%
```

Database Migrations

Strategy: EF Core

Production Migration Process:

1. **Pre-deployment Backup** (manual, antes del despliegue)
2. **Migration Execution** — el stage `RunMigrations` del pipeline está deshabilitado por defecto (`condition: false`); las migraciones se aplican manualmente con script idempotente EF Core o habilitando ese stage de forma controlada
3. **Data Validation** (post-migration SQL checks)
4. **Rollback Ready** (backup preserved for 30 days)

Creating New Migration

```
cd src/backend/DocumentIA.Functions

# 1. Add model changes (DocumentIA.Core/Data/*.cs)
# 2. Generate migration
dotnet ef migrations add AddExtractionTimeColumn \
  --project ../DocumentIA.Core \
  --startup-project .

# 3. Review generated migration
# File: src/backend/DocumentIA.Core/Data/Migrations/[timestamp]_AddExtractionTimeColumn.cs

# 4. Test locally
dotnet ef database update

# 5. Commit
git add src/backend/DocumentIA.Core/Data/Migrations/
git commit -m "Migration: Add ExtractionTime column"
```

Rollback Procedure

```
# If migration fails mid-deploy:
# 1. Stop Functions app
az functionapp stop --resource-group SRBRGDOCSAIPROD --name srbappprodocai

# 2. Restore database from backup
az sql db restore --server YOUR_SERVER --database DocumentIA \
  --backup-name BACKUP_NAME \
  --resource-group SRBRGDOCSAIPROD

# 3. Revert code to previous version
git checkout v<previous-version>

# 4. Re-run pipeline (without migrations this time)

# 5. Restart Functions
az functionapp start --resource-group SRBRGDOCSAIPROD --name srbappprodocai
```

Rollback Procedures

Automatic Rollback (if enabled)

Trigger Conditions:

- Pipeline fails (build, test, deployment)
- Smoke tests fail (errors > 5%)
- P95 latency > 100% vs baseline
- Application Insights errors > 10%

Process: Pipeline executes rollback stage automatically (see RUNBOOK_INCIDENTES_PRODUCCION.md for details)

Manual Rollback (Operator-initiated)

When to use: Undiscovered critical bug, data corruption, unacceptable performance

Steps (15-20 minutes):

1. Stop Current Deployment

```
az functionapp stop --resource-group SRBRGDOCSAIPROD --name srbappprodocai
az appservice web stop --resource-group SRBRGDOCSAIPROD --name srbwebadminprodocai
```


2. Restore Database (if needed)

```
# Identify backup
az sql db backup list --server YOUR_SERVER --database DocumentIA --resource-group SRBRGDOCSAIPROD

# Restore
az sql db restore --server YOUR_SERVER --database DocumentIA-Restored \
  --backup-name ... \
  --resource-group SRBRGDOCSAIPROD

# Swap: Archive failed DB, promote restored
```

3. Rollback Code

```
git checkout v<previous-version>
# Re-deploy via pipeline
```

4. Validation

```
./tests/smoke_e2e.ps1 -Environment Production
# Verify metrics normalized
```

Hotfix Process

When to Use Hotfix

- Critical bug affecting production (P1)
- Bug not caught in RC (test gap)
- Urgent security fix
- Data integrity issue

Hotfix Workflow (4-6 hours)

1. Create Hotfix Branch

```
git checkout main
git checkout -b hotfix/v<version>

# Minimal fix only (3-5 lines)
# No refactoring, no improvements
```

2. Expedited Code Review (30 min)

- 1 senior reviewer OK (vs 2 for normal release)
- Security review if applicable

3. Build & Deploy (30-45 min)

```
# Build only
# Run critical tests only
# Deploy to production
# Version: v<version> (PATCH bump)
```

4. Intensive Monitoring (2-4 hours)

- If fails immediately: automatic rollback to la versión previa
- If succeeds: Declare complete, close ticket

Communication Plan

Pre-Release (3 Days Before)

```
**Release Announcement: v<version>**
**When:** <fecha/hora> UTC
**What's new:**
- (resumen de cambios)
**Questions?** Contact Tech Lead
```

Pre-Deployment (24 Hours Before)

```
**Deployment Window**
**Release:** v<version>
**Start:** <fecha/hora> UTC
**Duration:** 30-45 minutes
**Expected Impact:** Latency spikes < 2 min
**Rollback:** Auto-enabled
```

During Deployment (Every 15 min)

```
T+0: Deployment started
T+2: Build complete
T+20: DB migrations done (si aplica)
T+30: Functions deployed
T+40: v<version> live
```

Post-Deployment (24 Hours After)

```
**Release v<version> Successful**
**Status:** Production, 24+ hours stable
**Metrics:** P95 latency <valor>
**Issues:** None
```

Post-Release Validation

24-Hour Monitoring

KQL Query (run hourly):

```
customMetrics
| where timestamp > ago(1h)
| where name startswith "DocumentIA"
| summarize
    p95_total_ms=percentile(iff(name == "DocumentIA.Duracion.Total", value, real(null)), 95),
    error_count=sum(iff(name == "DocumentProcessed" and customDimensions["EstadoFinal"] == "ERROR", 1, 0))
by bin(timestamp, 15m)
```

Thresholds:

Metric	Threshold	Action
P95 Total Duration	> 60s	Investigate
Error Rate	> 5%	Page on-call
CU Circuit Open	> 0	Monitor escalation

24-Hour Sign-Off

Release v<version> approved for sustained production

Metrics: P95 <valor>, Error <valor>, No incidents

Signed: Tech Lead | Date: <fecha> UTC

Appendix: Commands Reference

```
# Version management
git tag -l "v1.*" | sort -V
git tag -a v<version> -m "Release v<version>"

# Deployment control (parámetro targetEnvironment: dev | pre | prod)
az pipelines run --name azure-pipelines --parameters targetEnvironment=prod

# Validation
./tests/smoke_e2e.ps1 -Environment Production
```

Validación:

- Release strategy verificado contra configuración del pipeline
- Procedimientos basados en Azure DevOps + EF Core migraciones
- Rollback procedures incluyen restauración de backups
- Hotfix process acelerado para incidencias P1
- Communication plan con plantillas de comunicación
- Post-release validation con métricas específicas